PCS3858: LABORATÓRIO DE SISTEMAS EMBARCADOS

Especificação do Projeto Dirigido:
Dispositivo Vestível para Monitoramento de Saúde

Adrian Matheus Oliveira	NUSP: 11870944
Gabriel Pereira de Carvalho	NUSP: 11257668
Henrique Eduardo dos Santos de Souza	NUSP: 13679972



1 INTRODUÇÃO AO PROJETO

Problema e Ideia Central: O projeto propõe um sistema embarcado vestível para monitoramento de sinais fisiológicos (frequência cardíaca, estimativa de oxigenação e temperatura cutânea) e detecção de possíveis quedas, voltado ao acompanhamento de idosos e pessoas com necessidade de supervisão semelhante. O sistema não se propõe a ser um equipamento de diagnóstico médico, mas sim uma ferramenta de acompanhamento e apoio à supervisão de cuidadores e familiares.

Escopo do MVP (Versão Básica Fundamental):

- Atendimento a um único usuário;
- Operação em rede local (Wi-Fi), sem dependência de serviços em nuvem;
- Sem sensores sofisticados/complexos (ECG, pressão arterial, glicemia, etc.) (reservados para evoluções futuras).
- Sem funcionalidades extras (GPS, conectividade celular, diagnóstico automatizado ou integração hospitalar) (reservados para evoluções futuras).

O MVP inclui o desenvolvimento de dois subsistemas: o dispositivo vestível (aquisição, processamento inicial e identificação de eventos) e a central de acompanhamento baseada em Raspberry Pi (histórico, interface gráfica, armazenamento local e apresentação de alertas).

Para o desenvolvimento do MVP, foi escolhida a adoção de uma metodologia em cascata com validação incremental por módulo cujas etapas são detalhadas na seção 6 deste documento. A sequência de tarefas foi definida com uma abordagem top-down para decomposição e definição da arquitetura, combinada com prototipação iterativa para possibilitar o refinamento gradual das partes do projeto.

No espírito da metodologia em cascata, o processo de desenvolvimento descrito nessa especificação segue etapas sucessivas: requisitos, especificação, arquitetura, implementação dos componentes e integração com testes individuais por módulo antes da integração completa. O complemento com prototipação iterativa busca reduzir incertezas ligadas a contato óptico, fixação mecânica, posicionamento de sensores, falsos alarmes, estabilidade de comunicação, consumo de energia e usabilidade, através de três grandes fases: (1) versão de bancada para validar sensores e firmware isoladamente; (2) integração de comunicação, armazenamento e interface; (3) montagem vestível final.

2
ESPECIFICAÇÃO DE REQUISITOS

Requisitos Funcionais:

- **RF01:** Atualizar a telemetria na interface a cada 5 segundos, sem confundir esse intervalo com a taxa interna de amostragem dos sensores;
- **RF02:** Apresentar um alerta na central em até 5 segundos após confirmação pelo ESP32 (com rede disponível, excluídas janelas de análise/cancelamento);
- **RF03:** Indicar perda de comunicação após 10 segundos sem mensagens, mantendo sinalização local (LED/buzzer) quando a central estiver inacessível;
- **RF04:** Identificar eventos compatíveis com possíveis quedas a partir de dados do acelerômetro e giroscópio, permitindo cancelamento do alerta, antes do envio à central, em casos de falso positivo;
- **RF05:** Registrar dados fisiológicos e eventos detectados para consulta posterior, mantendo sequência e instante de aquisição;
- **RF06:** Disponibilizar interface web com dados atuais, histórico e estados de alerta;
- **RF07:** Permitir acesso autenticado aos dados armazenados;
- **RF08:** Detectar e sinalizar amostras inválidas, ausência de dados e falhas de comunicação;
- **RF09:** Realizar recuperação automática após interrupções temporárias de comunicação;
- **RF10:** Manter buffer limitado para preservar eventos importantes durante desconexões;
- **RF11:** Garantir autonomia mínima de 6 horas, dimensionada a partir do consumo medido (incluindo picos de transmissão).

Requisitos Não Funcionais:

- **RNF01:** Baixo consumo de energia e dimensões reduzidas visando a conveniência do usuário;
- **RNF02:** Baixo custo total de fabricação, considerando componentes e montagem;
- **RNF03:** Operação estável e facilidade de manutenção;
- **RNF04:** Segurança da comunicação e proteção dos dados armazenados (sem depender exclusivamente da rede como proteção);
- **RNF05:** Autenticação obrigatória no acesso à interface da central;
- **RNF06:** Uso eficiente dos recursos computacionais (memória e processamento do ESP32/Raspberry Pi).

O processamento dos sinais será realizado inicialmente no ESP32 do dispositivo vestível. O firmware realizará leituras periódicas, filtragem básica, identificação de amostras inválidas, cálculo ou estimativa dos parâmetros monitorados e aplicação das regras de detecção. Os sensores poderão possuir taxas internas de amostragem superiores à taxa de atualização apresentada na interface, de modo que o intervalo de atualização da aplicação não seja confundido com a frequência de aquisição utilizada pelos algoritmos de processamento.

A detecção de possíveis quedas será realizada por meio da combinação de informações do acelerômetro e do giroscópio. A primeira versão deverá considerar, de forma combinada, eventos como picos de aceleração, alterações bruscas de orientação e redução da movimentação após o evento. Após a identificação de uma situação compatível com queda, o dispositivo poderá solicitar confirmação do usuário por um intervalo configurável. Na ausência de confirmação ou cancelamento, o evento será encaminhado para a central como possível emergência. O objetivo inicial é reduzir falsos positivos por meio de regras simples e determinísticas, deixando métodos mais sofisticados de classificação ou aprendizado de máquina para versões futuras.

Propõe-se enviar os dados por MQTT sobre Wi-Fi. Cada mensagem conterá identificador do dispositivo, número de sequência, instante de aquisição, valores dos sensores, estado do dispositivo e indicadores de validade. O envio periódico de telemetria será separado do envio prioritário de eventos de alerta. Após uma desconexão, o dispositivo tentará restabelecer a comunicação e poderá preservar temporariamente eventos importantes em um buffer limitado para posterior reenvio.

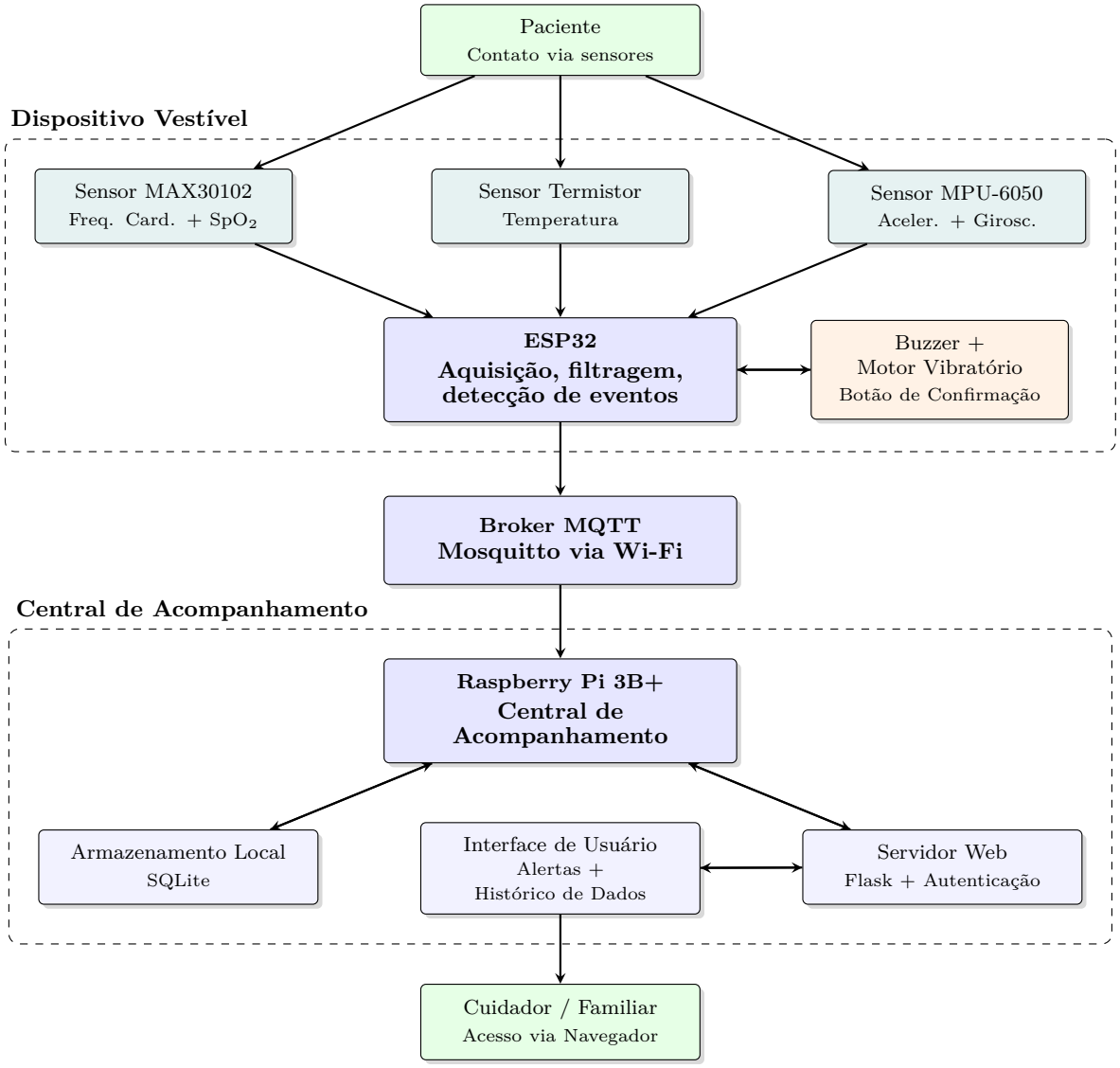
O ESP32 foi escolhido como plataforma principal por integrar processamento, interfaces de comunicação e conectividade sem fio em um dispositivo de baixo custo e dimensões reduzidas. Microcontroladores são particularmente adequados a sistemas embarcados desse porte por apresentarem interfaces, memória, processamento e recursos de comunicação suficientes para aplicações de pequeno e médio porte, além de baixa latência e baixo consumo energético.

Uma Raspberry Pi executará a recepção, o armazenamento local e uma interface web acessível por navegador. O Raspberry Pi 3B+ é a referência inicial para essa central. A tela apresentará valores recentes, gráficos de tendência, última atualização, estado da comunicação e estados de atenção, incluindo possível queda, sinal inválido e dispositivo desconectado. O histórico deverá permanecer armazenado localmente para permitir a consulta das informações adquiridas ao longo do tempo.

A interface deverá priorizar simplicidade de uso, especialmente considerando que o usuário da central poderá ser um cuidador ou familiar sem conhecimentos técnicos. Deverá ser possível visualizar, pelo menos, os valores atuais, tendências históricas e eventos detectados. O acesso aos registros deverá ser autenticado, evitando que qualquer usuário conectado à rede local possa visualizar os dados.

3
ARQUITETURA DO PROJETO

Figura 1: Arquitetura do Projeto - Dispositivo Vestível e Central de Acompanhamento.



O fluxo de dados segue o caminho natural dos sinais: Usuário Paciente → Conjunto de Sensores → ESP32 (aquisição, filtragem, cálculo/estimativa, regras de detecção) → Comunicação MQTT via Wi-Fi (telemetria periódica separada de eventos prioritários de alerta) → Raspberry Pi (recepção, armazenamento, processamento de histórico) → Interface Web (visualização autenticada) → Usuário(s) Cuidador(es).

4 COMPONENTES E ESTIMATIVA DE CUSTOS

Tabela 1: Orçamento Proposto para um Dispositivo Vestível e uma Central.

Componente	**Valor (R$)**
Placa de desenvolvimento ESP32	já temos
Raspberry Pi 3B+	já temos
Sensor MAX30102	15,90
Sensor MPU-6050	22,90
Sensor Termistor	12,00
Motor vibratório	15,00
Bateria Li-Po 3,7 V 500 mAh	45,00
Módulo Carregador TP4056	12,00
Regulador e componentes de alimentação	20,00
Buzzer	já temos
LEDs	já temos
Botões de acionamento	já temos
Protoboard, fios e conectores	já temos
Material para pulseira/carcaça	já temos

Os valores apresentados na tabela de orçamento acima foram obtidos pelo grupo através de consultas aos principais sites de fornecedores de componentes disponíveis na internet, conforme sugestão dos professores. O custo estimado dos componentes adicionais para uma primeira unidade funcional encontra-se em aproximadamente R$ 200,00, considerando os componentes já disponíveis pelo grupo, o desenvolvimento do protótipo, considerando aquisição de componentes de reposição, eventuais substituições, conectores e materiais mecânicos.

Em uma etapa posterior, caso o protótipo seja validado, o uso de uma placa dedicada poderá reduzir dimensões, quantidade de conexões e custo unitário. A primeira versão, entretanto, deverá priorizar disponibilidade de componentes, velocidade de implementação e facilidade de testes, seguindo a recomendação de utilização de plataformas de prototipação antes da eventual otimização para um produto final.

O grupo já possui diversos dos componentes propostos para a realização do projeto, incluindo a placa de desenvolvimento ESP32 e a Raspberry Pi 3B+, e os integrantes pretendem autofinanciar a compra dos sensores e demais componentes restantes conforme indicado na tabela de orçamento acima.

5 FERRAMENTAS DE DESENVOLVIMENTO DE SOFTWARE E LINGUAGENS

Com base na arquitetura descrita (ESP32 + comunicação MQTT + Raspberry Pi com interface web autenticada e armazenamento local), as ferramentas e linguagens escolhidas pelo grupo para a implementação são:

No Dispositivo Vestível (ESP32):

- **Linguagem:** C++
- **Ambiente:** Arduino IDE ou PlatformIO (com framework Arduino-ESP32 ou ESP-IDF)
- **Bibliotecas de Sensores:** drivers/bibliotecas específicas do MAX30102 e MPU-6050 (via I²C) e de conversão para o termistor (via ADC+)
- **Biblioteca MQTT para ESP32:** PubSubClient

Na Central (Raspberry Pi):

- **Linguagem:** Python (backend, processamento de dados e broker/cliente MQTT)
- **Broker MQTT:** Mosquitto
- **Framework Web:** Flask (leve, adequado a uma interface local com autenticação simples) servindo páginas com HTML/CSS/JavaScript no front-end
- **Armazenamento local:** SQLite (banco de dados leve, adequado a um único usuário e operação local)
- **Biblioteca de Gráficos/Tendências no Front-End:** Chart.js

6 METODOLOGIA E CRONOGRAMA

Para a realização do projeto, o grupo optou pela adoção da metodologia de desenvolvimento em cascata mostrada na Figura 2.

Para o desenvolvimento dessas atividades foi proposto o cronograma apresentado na Tabela 2, seguindo as recomendações passadas na primeira aula e considerando o período de 12 semanas da disciplina.

Figura 2: Fases de Desenvolvimento do Projeto em Cascata.

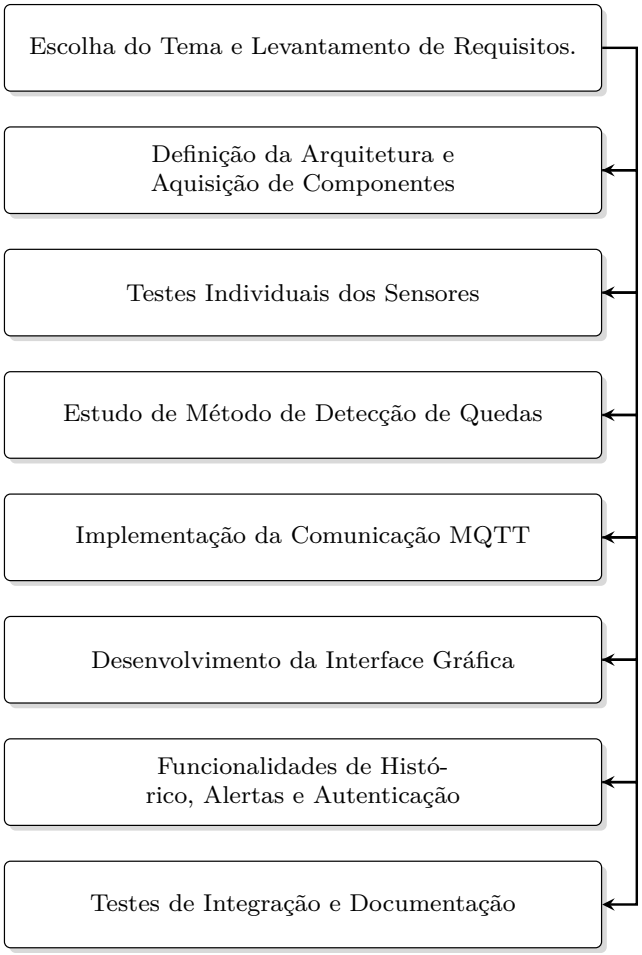


Tabela 2: Cronograma Proposto para Desenvolvimento do Projeto

Fase do Projeto	**Semanas 1-2**	**Semanas 3-4**	**Semans 5-6**	**Semans 7-8**	**Semans 9-10**	**Semanas 11-12**
Escolha do Tema						
Levantamento de Requisitos						
Definição da Arquitetura						
Aquisição de Componentes						
Testes Individuais dos Sensores						
Detecção de Quedas						
Comunicação MQTT						
Interface Gráfica						
Funcionalidade de Histórico						
Funcionalidade de Alertas						
Funcionalidade de Autenticação						
Testes de Integração						
Conclusão da Documentação						
Preparações para a Apresentação Final						